&

Full-Stack Natural Language Spreadsheet Analysis • Formula Engineering • Data Visualization • Cloud Ready

1. AI MODELS & API KEYS USED IN THIS PROJECT

LLM Tier	Provider	Exact Model Identifier	Python SDK	Env Variable	Operational Role in Application
PRIMARY	Google	`gemini-2.5-flash-preview-05-20`	google-genai==2.31.0	`GEMINI_API_KEY`	Answers all natural language questions, inspects column headers, generates Excel formulas (=SUM, =AVERAGE, =XLOOKUP, etc.), determines optimal chart type, and explains results.
FALLBACK	OpenAI	`gpt-4o-mini`	openai==3.33.0	`OPENAI_API_KEY`	Automatic fallback service. Engages seamlessly if Gemini key is not configured or if API limits / quota exceptions are encountered. Zero user-facing downtime.

2. COMPLETE TECHNOLOGY STACK & EXACT VERSIONS

Category	Component	Version	Function & Practical Responsibility
Language	Python	`3.11`	Underlying runtime execution environment for backend processing
Web Framework	FastAPI	`0.143.3`	High performance asynchronous REST API with automatic OpenAPI Swagger docs
ASGI Server	Uvicorn	`0.54.0`	Async server handling HTTP requests and WebSocket connections
Spreadsheet Core	Pandas	`3.0.6`	DataFrame loading, aggregations, filtering, group-by, min/max/mean computations
Numerical Arrays	NumPy	`2.4.6`	Vectorized numeric calculations powering Pandas analytical transformations
Excel File Engine	openpyxl	`3.1.5`	Direct read/write access to .xlsx files, sheet metadata, formatting, formula writing
Data Plotting	Matplotlib	`3.11.2`	Core plotting engine creating bar, line, pie, and scatter charts saved as PNG
Plot Styling	Seaborn	`0.13.2`	Statistical palette and theme styling applied over Matplotlib graphs
Database ORM	SQLAlchemy	`2.1.5`	Object-relational mapping models, query builder, and relationship links
SQL Database	SQLite (ai_excel.db)	`3.x (Built-in)`	Persistent database storing users, uploaded files, chat logs, and analysis history
Security & Hash	bcrypt	`5.0.0`	Direct password hashing with salt generation (avoids passlib 4.x bug)
Tokens & JWT	python-jose	`3.5.0`	Signed HMAC-SHA256 JWT generation and validation for session authentication
Validation	Pydantic & Settings	`2.14.1 / 2.16.0`	Pydantic v2 schemas for request validation and environment parsing (.env)
Client & Search	httpx	`0.28.1`	Async HTTP client used for web research queries and test client invocation
OTP Mailer	smtplib (Standard)	`Built-in`	Transmits 6-digit verification OTPs to user email via TLS-encrypted SMTP
Testing	pytest	`9.1.1`	Automated unit and integration test suite (15 passed tests across 6 modules)
Frontend UI	HTML5 + CSS3 + JS	`ES6 Vanilla`	Single Page App (SPA) featuring Chat, Uploads, 1-Click '■ Start All' run
Deployment 1	Render	`Cloud Blueprint`	PaaS hosting configured via render.yaml for automated web service build
Deployment 2	Vercel	`Serverless WSGI`	Edge serverless deployment configured via vercel.json + api/index.py

3. SYSTEM APPLICATION & DATA PROCESSING FLOW

Flow Steps: 1. **Register & OTP** (User signs up → 6-digit OTP emailed → verified) • 2. **JWT Authentication** (Login returns signed JWT token stored in browser) • 3. **Spreadsheet Ingestion** (Upload .xlsx → Pandas extracts worksheets, column names, datatypes, sample rows → saved in SQL) • 4. **Natural Language Query** (User asks question in chat) • 5. **LLM Routing** (Gemini 2.5 Flash detects intent: Formula / Calculation / Chart / Q&A) • 6. **Execution** (Formula generated & validated, Pandas computes aggregates, or Matplotlib generates PNG chart) • 7. **History Saved** (Result logged to SQLite database table analysis_history).

4. COMPLETE CODEBASE MAP — EVERY FILE, LOCATION & RESPONSIBILITY

Every module in the project is structured according to clean architecture principles: separation of Data Models, API Routes, Business Logic Services, Schemas, and Static Web Assets.

File Path / Location	Layer	Key Classes & Methods	Purpose & Implementation Details
`app/main.py`	**Entry Point**	app, lifespan(), mount()	Main FastAPI initialization, CORS middleware setup, router registration, static files mounting, and startup DB schema creation.
`app/config.py`	**Config**	Settings, settings	Environment variable loader using Pydantic Settings. Reads .env (API keys, DB URLs, secret tokens) with Vercel cloud detection.
`app/database.py`	**Database**	Base, engine, get_db()	SQLAlchemy engine creation, SessionLocal factory, declarative base, and request-scoped session dependency injection.
`app/models/user.py`	**ORM Model**	class User(Base)	Table 'users': Stores user id, email, hashed password, is_verified status, 6-digit OTP code, and expiration timestamp.
`app/models/excel_file.py`	**ORM Model**	class ExcelFile(Base)	Table 'excel_files': Stores uploaded file path, original name, detected sheet names, row counts, and JSON column schemas.
`app/models/chat_session.py`	**ORM Model**	class ChatSession(Base)	Table 'chat_sessions': Tracks chat threads linking authenticated users with specific uploaded Excel spreadsheets.
`app/models/chat_message.py`	**ORM Model**	class ChatMessage(Base)	Table 'chat_messages': Individual user and assistant conversation turns with timestamps, roles, and message content.
`app/models/analysis_history.py`	**ORM Model**	class AnalysisHistory(Base)	Table 'analysis_history': Structured query logs recording formulas generated, chart configurations, and summary statistics.
`app/schemas/*.py`	**Validation**	UserCreate, ChatQuery, ...	Pydantic v2 schemas (auth, excel, chat, formula) enforcing strict request payloads and serialized JSON API responses.
`app/services/llm_service.py`	**AI Logic**	generate_response(), fallback	Dual-provider LLM engine: Queries Gemini 2.5 Flash first via google-genai, with automated fallback to OpenAI GPT-4o-mini.
`app/services/excel_service.py`	**Spreadsheets**	inspect_excel(), get_sheet_df()	Loads .xlsx files via Pandas and openpyxl. Extracts headers, datatypes, sample rows, and performs Pandas calculations.
`app/services/formula_service.py`	**Formulas**	build_formula(), validate()	Prompts LLM with strict spreadsheet column constraints. Produces syntax-validated formulas (SUM, AVERAGE, XLOOKUP, IF).
`app/services/chart_service.py`	**Visuals**	create_chart(), render_png()	Generates Bar, Line, Pie, and Scatter plots via Matplotlib + Seaborn. Saves high-res PNG charts to the charts/ directory.
`app/services/auth_service.py`	**Security**	hash_password(), create_jwt()	Handles registration, password hashing via bcrypt, OTP verification, and JWT access token issuance.
`app/services/email_service.py`	**Mailing**	send_otp_email()	Sends branded HTML emails containing 6-digit activation OTP codes using smtplib via configured SMTP servers.
`app/services/chat_service.py`	**Orchestration**	process_message()	Central dispatcher: parses questions, classifies intent (formula, chart, aggregation, or generic), calls services, saves history.
`app/routers/*.py`	**REST API**	auth, excel, chat, formula	FastAPI route controllers exposing endpoints for user auth, file upload, sample dataset loading, chat, and formula generation.
`app/static/index.html`	**Frontend**	SPA Interface	Single Page Application UI: Upload zone, Chat view, Dynamic charts panel, and the 1-click '■ Start All (Demo Run)' button.
`app/static/js/app.js`	**Frontend JS**	runStartAllDemo(), API calls	Handles authentication state, file uploads, chat message streaming, chart rendering, and automated demo execution.
`start.bat / start.ps1`	**Launcher**	1-Click Launch Scripts	Automated Windows launchers that verify Python, set up virtualenv, install dependencies, and launch browser at localhost:8000.
`render.yaml / vercel.json`	**Deployment**	Cloud Infrastructure	Render Blueprint specification for containerless deployment + Vercel serverless functions wrapper (api/index.py).

5. CORE CODE LOGIC FROM SCRATCH — HOW THE ENGINE WORKS

Below are the actual core architectural code implementations powering LLM reasoning, formula generation, Excel processing, and chart rendering.

5.1 Dual-Model LLM Service Implementation (app/services/llm_service.py)

```
from google import genai
from openai import OpenAI
from app.config import settings

def call_llm(prompt: str, system_instruction: str = "") -> str:
# 1. Primary Model: Google Gemini 2.5 Flash
if settings.GEMINI_API_KEY:
try:
client = genai.Client(api_key=settings.GEMINI_API_KEY)
response = client.models.generate_content(
model="gemini-2.5-flash-preview-05-20", # Exact Primary Model
contents=prompt,
config={"system_instruction": system_instruction} if system_instruction else None
)
if response.text:
return response.text.strip()
except Exception as e:
print(f"[LLM] Gemini attempt failed: {e}. Switching to OpenAI fallback...")

# 2. Fallback Model: OpenAI GPT-4o-mini
if settings.OPENAI_API_KEY:
try:
client = OpenAI(api_key=settings.OPENAI_API_KEY)
messages = [{"role": "system", "content": system_instruction}] if system_instruction else []
messages.append({"role": "user", "content": prompt})
resp = client.chat.completions.create(model="gpt-4o-mini", messages=messages, temperature=0.2)
return resp.choices[0].message.content.strip()
except Exception as e:
print(f"[LLM] OpenAI fallback also failed: {e}")

return "Error: No active LLM key or providers unreachable."
```

5.2 Excel Processing & Metadata Extraction (app/services/excel_service.py)

```
import pandas as pd
import openpyxl

def inspect_spreadsheet(file_path: str) -> dict:
wb = openpyxl.load_workbook(file_path, read_only=True, data_only=True)
sheets_info = {}
for sheet_name in wb.sheetnames:
df = pd.read_excel(file_path, sheet_name=sheet_name, nrows=100)
columns_meta = []
for col in df.columns:
columns_meta.append({
"column_name": str(col),
"data_type": str(df[col].dtype),
"is_numeric": bool(pd.api.types.is_numeric_dtype(df[col])),
"sample_values": [str(v) for v in df[col].dropna().head(3).tolist()]
})
sheets_info[sheet_name] = {"columns": columns_meta, "total_rows": len(df)}
return sheets_info
```

5.3 Formula Generation with Column Constraints (app/services/formula_service.py)

```
def generate_formula(user_query: str, columns_meta: list) -> dict:
sys_prompt = "You are an Excel Formula Expert. Generate valid formulas based ONLY on provided columns."
user_prompt = f"""Columns available: {columns_meta}
User Request: '{user_query}'
Return JSON with:
1. 'formula': Excel formula starting with '=' (e.g., =SUMIF(C2:C100, \">500\", D2:D100))
2. 'explanation': Plain English explanation of how the formula calculates the result.
3. 'target_column': Suggested column to place this formula in.
Never hallucinate or create columns not present in the spreadsheet."""
raw = call_llm(user_prompt, system_instruction=sys_prompt)
return parse_json_safely(raw)
```

5.4 Chart Generation Engine (app/services/chart_service.py)

```
import matplotlib
matplotlib.use('Agg') # Thread-safe non-GUI backend
import matplotlib.pyplot as plt
import seaborn as sns

def render_chart(df: pd.DataFrame, chart_type: str, x_col: str, y_col: str, title: str, output_path: str):
plt.figure(figsize=(8, 4.5))
sns.set_theme(style="whitegrid")
if chart_type == "bar":
sns.barplot(data=df.head(15), x=x_col, y=y_col, palette="Blues_r")
plt.xticks(rotation=45, ha='right')
elif chart_type == "line":
sns.lineplot(data=df, x=x_col, y=y_col, marker="o", color="#1565C0")
elif chart_type == "pie":
data = df.groupby(x_col)[y_col].sum().head(7)
plt.pie(data.values, labels=data.index, autopct='%1.1f%%', colors=sns.color_palette("Set2"))
plt.title(title, fontsize=12, fontweight='bold', pad=10)
plt.tight_layout()
plt.savefig(output_path, dpi=150)
plt.close()
```

6. REST API SPECIFICATION (FASTAPI SWAGGER / OPENAPI)

HTTP Method	Route Endpoint	Auth Required	Payload / Parameters	Response Description
`POST`	/api/auth/register	No	`{email, password}`	Registers user & dispatches 6-digit OTP code to email
`POST`	/api/auth/verify-otp	No	`{email, otp_code}`	Validates OTP, marks account verified and active
`POST`	/api/auth/login	No	`{email, password}`	Authenticates password via bcrypt, returns signed JWT token
`POST`	/api/excel/upload	Bearer JWT	`multipart/form-data (.xlsx)`	Parses sheet names, column headers, datatypes, row counts
`GET`	/api/excel/load-sample	Bearer JWT	`None (query param)`	Loads bundled 100-row sample_sales_data.xlsx immediately
`POST`	/api/chat/query	Bearer JWT	`{file_id, question, session_id}`	Generates answer, formula, chart URL, or Pandas summary
`POST`	/api/formula/generate	Bearer JWT	`{file_id, natural_language_query}`	Produces Excel formula + syntax check + plain English explanation
`GET`	/api/history	Bearer JWT	`None`	Returns chronological history of all questions and generated results
`GET`	/charts/{filename}	No	`Chart file path`	Serves rendered Matplotlib PNG visualization to the UI

7. LOCAL SETUP & RUN INSTRUCTIONS

Step 1: Clone Repository

```
git clone https://github.com/Pavanadithya12/excel-ai-assistant.git
```

Step 2: Automated 1-Click Launch (Recommended for Windows)

Double-click **start.bat** (or run `.\start.ps1`). Automatically initializes virtual environment, installs dependencies, loads default settings, starts Uvicorn at `http://localhost:8000`, and launches your web browser.

Step 3: Manual Startup

```
python -m venv .venv  |  .venv\Scripts\activate  |  pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Step 4: 1-Click Full Automation Demo in UI

Once the web UI opens, click the prominent ■ **Start All (Demo Run)** button. The system will auto-login, load the sample sales workbook, execute 3 sample queries (aggregations, Excel formulas, and a bar chart), and display results immediately without typing!

8. CLOUD DEPLOYMENT & AUTOMATED TEST VERIFICATION

Render Deployment (render.yaml Blueprint):

1. Log in to **dashboard.render.com** → Click **New** → **Blueprint**.
2. Connect your GitHub repository (`Pavanadithya12/excel-ai-assistant` or `revalsysprashant/AI-Excel-Assistant`).
3. Render automatically reads **render.yaml** and configures the web service.
4. In the Render Environment tab, add your **GEMINI_API_KEY** and **SECRET_KEY**, then click **Apply**.

Vercel Deployment (vercel.json Serverless):

Run `vercel` then `vercel --prod`. Routed through `api/index.py` using FastAPI WSGI adapter.

Automated Test Suite (15/15 Passed):

Execute `pytest tests/ -v`. Verifies user registration, bcrypt security, OTP lifecycle, Excel loading, formula validation, Pandas summary calculation, chart generation, and API router responses.